UNIVERSITY OF INFORMATION TECHNOLOGY – VNU HCMC

PROJECT REPORT

BUILDING A REAL-TIME DATA LAKEHOUSE ARCHITECTURE FOR INTELLIGENT TRANSPORTATION SYSTEMS ON A KUBERNETES CLUSTER

Course: Distributed Databases and Big Data – IS211.Q22 & IS405.Q23 Supervisor: M.Sc. Nguyễn Hồ Duy Trí
Students: Ngô Tiến Sỹ (23521367), Nguyễn Văn Nam (23520982) Ho Chi Minh City, 2026

Abstract

Intelligent transportation systems must continuously ingest and aggregate event data while allowing analytical logic to evolve without interrupting serving queries. This study designs and evaluates **Continux**, a real-time data lakehouse architecture deployed on a resource-constrained three-server K3s cluster. The pipeline uses the March 2026 NYC TLC Yellow Taxi dataset, converts Parquet records to JSONL, replays events through Vector and Redpanda, processes streaming SQL in RisingWave, delivers results to Apache Iceberg on MinIO, manages deployment with Argo CD, and observes runtime behavior with VictoriaMetrics and Grafana.

Following Design Science Research, the study evaluates an architectural artifact through one controlled replay and a Blue/Green materialized-view (MV) cutover scenario. The converted source dataset contains `3,952,451` records; the measured replay was deliberately stopped when the public materialized view reached `69` zones and `986` trips and therefore does not represent full-dataset processing. Before cutover, the public/blue views yielded `69/986`, whereas the green view, applying additional data-quality filters, yielded `69/978`. The runbook records a `0.145226 s` duration for `ALTER MATERIALIZED VIEW ... SWAP WITH ...`; the query loop observed no query errors, RisingWave components did not restart, and the public MV reflected green logic after the swap.

The results demonstrate **observed query-layer zero-downtime** within the experimental window and the feasibility of running a GitOps-observed streaming lakehouse on a small K3s cluster. This study does not claim independently verified end-to-end exactly-once processing, because the available evidence does not reconcile offsets, duplicates, or losses across the complete path. It also does not claim that the Iceberg sink dependency automatically followed the green logic after the MV name swap.

Keywords: Data Lakehouse, stream processing, Materialized View, Blue/Green cutover, RisingWave, Apache Iceberg, GitOps, K3s, ITS.

Tóm tắt

Các hệ thống giao thông thông minh cần tiếp nhận và tổng hợp dữ liệu phát sinh liên tục, đồng thời cần thay đổi logic phân tích mà không làm gián đoạn khả năng truy vấn đang phục vụ. Nghiên cứu này thiết kế và đánh giá **Continux**, một kiến trúc Data Lakehouse thời gian thực chạy trên cụm K3s ba server có tài nguyên giới hạn. Pipeline sử dụng dữ liệu NYC TLC Yellow Taxi tháng `2026-03`, chuyển đổi Parquet sang JSONL, phát sự kiện bằng Vector, trung chuyển qua Redpanda, xử lý bằng RisingWave streaming SQL, ghi kết quả theo Apache Iceberg trên MinIO, quản lý triển khai bằng Argo CD và quan sát qua VictoriaMetrics/Grafana.

Theo phương pháp Design Science Research, nghiên cứu xây dựng artifact kiến trúc và đánh giá qua một lượt replay có kiểm soát cùng kịch bản Blue/Green materialized view (MV) cutover. Dataset nguồn sau chuyển đổi có `3.952.451` dòng; lượt thực nghiệm được dừng có chủ đích sau khi public MV đạt `69` vùng và `986` chuyến, không phải phép xử lý toàn bộ dataset. Trước cutover, public/blue MV có kết quả `69/986`, còn green MV áp dụng bộ lọc chất lượng dữ liệu đạt `69/978`. Runbook ghi duration của lệnh `ALTER MATERIALIZED VIEW ... SWAP WITH ...` là `0,145226 s`; vòng lặp truy vấn không ghi nhận lỗi, các thành phần RisingWave không restart và public MV sau swap phản ánh logic green.

Kết quả chứng minh tính khả thi của **zero-downtime quan sát được ở lớp truy vấn** trong cửa sổ thực nghiệm, đồng thời cho thấy một pipeline lakehouse có thể vận hành trên cụm K3s nhỏ với GitOps và quan sát tập trung. Nghiên cứu không tuyên bố đã kiểm chứng Exactly-Once end-to-end, vì evidence hiện có không bao gồm phép đối chiếu offset, duplicate hoặc loss trên toàn đường dữ liệu; cũng không kết luận Iceberg sink tự chuyển phụ thuộc theo logic green sau thao tác đổi tên MV.

1. Introduction

1.1 Problem context

Urban mobility data arrives continuously and has immediate operational relevance: each trip contributes pickup location, distance, fare, and zone-demand observations. An Intelligent Transportation System (ITS) – a transport

platform that uses real-time data to coordinate dispatch, analytics, and planning – requires more than delayed batch analysis; operators also need near-real-time aggregates and a way to modify analytical rules when data-quality or business requirements evolve. "Near-real-time" here means that results must reflect the most recent events within seconds to tens of seconds rather than after a scheduled batch run.

In batch processing, the system collects a body of data and computes results on a schedule, so an analytical view commonly represents the past after a planned delay. Stream processing instead treats each newly emitted record as an *event* – a message that can update a served result the moment it arrives. A key abstraction that makes this possible at the analytical layer is the *materialized view* (MV): a query result that the system maintains incrementally as input changes, so readers query the maintained aggregate rather than recomputing from raw records. This distinction matters for ITS: a changing zone-demand aggregate is operationally useful only if it can be observed soon enough and if its definition can be modified safely while it is being served.

The lakehouse model is an architecture that combines the flexibility of a *data lake* (open files on inexpensive object storage) with the managed table semantics of a *data warehouse*, and was formally proposed by Armbrust et al. (2021). Stream processing adds further components to the operational path: a *broker* (intermediate layer that stores and routes messages), a computation engine, a *state store* (which holds intermediate state across events), a *sink* (output that writes to a downstream system), and an observability layer. The Ursa paper (Merli et al., 2025) identifies the cost and complexity that arise when streaming and lakehouse infrastructure are separated, and proposes a Kafka-compatible lakehouse-native engine. At the analytical application layer, a practical question remains widely unanswered: can the definition of a serving aggregate be changed without observed query failures during the transition?

1.2 Motivation

Continux studies the question above in a setting that is reproducible in a laboratory: a lightweight Kubernetes cluster, the public NYC TLC taxi dataset, and standard open-source components. The chosen Kubernetes distribution is K3s – a lightweight Kubernetes distribution maintained by SUSE that can run on resource-constrained machines – which allows the project to exercise core Kubernetes concepts without large-scale cloud infrastructure. The ITS motivation also relates to the United Nations Sustainable Development Goals (SDG): SDG 8 through efficient mobility-supporting digital infrastructure, SDG 9 through data infrastructure innovation, and SDG 11 through observable urban mobility analytics (United Nations, n.d.-a, n.d.-b, n.d.-c). These connections motivate the application domain; they do not constitute a quantified SDG impact evaluation.

Ursa primarily addresses streaming-engine and ingestion architecture. Continux does not reproduce Ursa's cost benchmark; it instead examines the *operational layer* above ingestion, comprising GitOps-managed deployment, observability dashboards, and a RisingWave materialized-view name swap used to evolve public analytical logic, which is detailed in Chapter 2.

1.3 Objectives and scope

The study has four coordinated objectives that move from design to evaluation. *First*, design a streaming lakehouse pipeline that runs end-to-end from NYC TLC data to Apache Iceberg objects on object storage. *Second*, deploy that pipeline on a high-availability (HA) K3s cluster of three servers with a clean separation between a *data plane* (the components that process data), a *control/observability plane*, and a *quorum-only* node that participates in cluster consensus voting without hosting data workloads. *Third*, execute a Blue/Green cutover at the public materialized-view query layer and observe transition duration, query errors, and workload status in that window. *Fourth*, evaluate the artifact strictly from collected repository evidence and supplied dashboard captures, without extending conclusions beyond what was measured.

The processed source is the public NYC TLC Yellow Taxi dataset for `2026-03`, joined with the Taxi Zone Lookup table. The Parquet input – a columnar compressed format optimized for analytics – is converted to `3,952,451` JSONL records. The reported experiment deliberately stops the Vector replay process after the public MV reaches `986` trips; the final MV therefore reflects a sample rather than the full converted file. Cutover conclusions apply to the public query name `mv_zone_stats`; the study does not claim that the downstream Iceberg sink automatically migrates to green logic as a result of the materialized-view name swap.

1.4 Research questions

Table 1.1: Research questions and available evidence

ID	Question	Evidence	Bounded conclusion
RQ1	Can public analytical logic be switched without observed query errors during the swap window?	Runbook duration `0.145226 s`; query errors `0`.	Yes, at query layer in the observed window.
RQ2	Does the pipeline generate observable lakehouse output?	MinIO lists Parquet data, delete objects, and metadata.	Yes, after replay.
RQ3	Can the constrained cluster run the designed scenario?	`3/3` nodes, `5/5` PVCs, `23/23` workloads.	Yes, for this scenario.
RQ4	Has end-to-end exactly-once behavior been verified?	No offset/duplicate/loss reconciliation.	Not established.

1.5 Contributions and organization

Continux contributes four interrelated outcomes. First, a reproducible streaming lakehouse artifact deployed on a three-server K3s cluster using GitOps – an operating model in which desired state is declared in Git and an agent (here Argo CD) continuously enforces it. Second, a Blue/Green materialized-view swap scenario measured while a public query loop runs continuously. Third, a set of observability dashboards together with explicit interpretation guidelines for cutover, resources, and data integrity. Fourth, an evidence-bounded analysis that clearly separates *documented platform capabilities* from *measured outcomes at this artifact*.

Chapter 2 presents the theoretical background on lakehouse storage, stream processing, materialized views, GitOps, observability, and the Ursa paper. Chapter 3 describes the Design Science Research method, the Continux architecture, K3s topology, SQL objects, and Blue/Green design. Chapter 4 reports experimental results, including a source-data quality analysis, replay ingestion, cutover, and resource observations. Chapter 5 discusses results, limitations, and conclusions. The appendices preserve core SQL, configuration excerpts, evidence indexing, and a detailed terminology table.

2. Theoretical Background

2.1 Lakehouse and Apache Iceberg

A *data warehouse* traditionally places data into closely governed tables for stable analytical serving: schemas are controlled, ETL (*Extract – Transform – Load*) operations clean and shape data before it is loaded, and engines query the curated tables. A *data lake*, in contrast, stores diverse files (CSV, JSON, Parquet, Avro, . . .) on inexpensive object storage and does not require a shared schema; ingest becomes easier but table-level governance can be lost. The *lakehouse* pattern combines the two directions: data remains in open formats on a lake, while a table layer adds *metadata*, versioned state, and transactional semantics that satisfy ACID properties (*Atomicity, Consistency, Isolation, Durability*), so different engines can read a single consistent analytical result (Armbrust et al., 2021).

Object storage is a storage layer in which data is accessed by *key* through an HTTP API – typically compatible with the Amazon S3 interface – rather than through traditional file paths; each object is a blob of bytes with attached metadata. In this artifact, MinIO supplies an S3-compatible endpoint for both checkpoints and Iceberg tables. Apache Iceberg is not a query processor but a *table format* – a specification that identifies which data files make up a table, which *snapshot* (a table's state at a point in time) is current, and which *manifest* files (intermediate metadata listing data files) describe table evolution (Apache Software Foundation, n.d.). Iceberg distinguishes two delete object types: *equality-delete*, which marks rows for deletion by key value, and *position-delete*, which marks rows by position within a file. Both mechanisms support upsert-style updates. Consequently, when the report states that "lakehouse output exists," the concrete evidence consists of Parquet data files and Iceberg metadata objects produced by the sink on MinIO; it does not assert that MinIO itself executes SQL.

2.2 Stream processing, state, and materialized views

In the pipeline of this study, an *event* is one taxi-trip record replayed as a JSON message. Stream processing updates results as such events arrive rather than waiting for a complete batch. The abstraction that serves near-real-time results here is the *materialized view* (MV): while an ordinary view is only a query definition re-evaluated at read time, a materialized view is a derived query result that the system maintains incrementally as inputs change; readers always observe the current aggregate without recomputing from raw records. For the taxi dataset, the MV is grouped by `borough` and `zone` and exposes the fields `trip_count`, `total_fare`, and `avg_distance`.

The lookup join and grouped aggregation both require *state* – intermediate information that is retained across many events to keep running sums and counts per zone correct. A *checkpoint* is a deliberately persisted snapshot of that

state so an engine has a point from which processing can resume or recover after a disruption. Regarding semantics, a pipeline can be *at-least-once* (each event is processed at least once, possibly duplicating it), *at-most-once* (events may be lost), or *exactly-once* (each event affects the result exactly once). A complementary property is *idempotence* – applying the same operation many times yields the same outcome – which is often used at sinks to turn at-least-once delivery into effective exactly-once results. RisingWave is configured here to store checkpoint and state material in the MinIO `rw-checkpoint` bucket. That configuration explains where state is placed, but by itself it does not establish recovery behavior or end-to-end exactly-once delivery for this artifact.

2.3 Kafka-compatible event streaming and RisingWave

Kafka established the now-standard partitioned distributed-log model for streaming. In this model, a *topic* is a named event stream (a "pipe" that holds messages of one kind), a *partition* is a slice of a topic that provides local ordering and parallelism, and an *offset* is the sequence position of a message within a partition (Kreps et al., 2011). The *producer* writes messages to a topic, while a *consumer* reads them; consumers track the offsets they have processed so they can resume without re-reading. Continux uses Redpanda – a C++ broker that is wire-compatible with the Kafka API – as the broker that separates the Vector replay producer from the RisingWave consumer (Redpanda Data, n.d.). This intermediate layer allows replay emission and streaming computation to progress at different rates: if RisingWave processes more slowly than Vector emits, messages remain buffered in the topic.

The topic is named `nyc-taxi-events` and is provisioned with three partitions and one replica. Three partitions characterize stream division and parallelism, while one replica means this run does not evaluate broker-node loss under a production high-availability arrangement – if a broker node is lost, no in-topic copy survives. Because the collected evidence also does not reconcile offsets across the complete path, the report does not present exactly-once behavior as an experimentally verified end-to-end result.

RisingWave reads the Kafka-compatible event source, joins S3-compatible lookup data, maintains MVs, and writes to an Iceberg sink. RisingWave documentation identifies an Iceberg sink option for exactly-once delivery that is enabled by default while sink decoupling is enabled (RisingWave Labs, n.d.-c). "Sink decoupling" decouples the sink's write schedule from the MV computation cadence to improve stability. This is a documented connector configuration capability, not an independently measured end-to-end result for this artifact.

2.4 Blue/Green switching and dependencies

Blue/Green deployment is a deployment pattern in which two versions of the same service are maintained side by side: *blue* is the version currently serving real traffic, while *green* is a candidate replacement that is already prepared but is not publicly reachable. Once the candidate is judged ready, a *cutover* switches the public access point from blue to green; when the change is purely a rename, that operation is called a *swap*. Continux applies this pattern to MV definitions rather than to Kubernetes pods: `mv_zone_stats` is the stable public name that clients continue to call after the cutover; `mv_zone_stats_blue` (baseline) retains all joined records, and `mv_zone_stats_green` filters records with negative `fare_amount` or `trip_distance`.

RisingWave specifies `ALTER MATERIALIZED VIEW ... SWAP WITH ...` as a materialized-view name-swap operation (RisingWave Labs, n.d.-b). "Atomic" in this context concerns the public name operation as observed within the engine – meaning the rename appears instantaneous to a client and exposes no intermediate state; it does not by itself establish that every dependent consumer or sink also changes its upstream definition. RisingWave's streaming-job modification guidance treats downstream dependencies such as sinks separately (RisingWave Labs, n.d.-a); consequently, this study's cutover finding is bounded to the public query endpoint and does not extend to the downstream Iceberg sink.

2.5 GitOps and observability

OpenGitOps defines four principles for desired system state: it must be *declarative* (describing "what" rather than "how"), *versioned* in Git, automatically *pulled* by an agent, and continuously *reconciled* – the agent compares live resources with the desired configuration and drives them back toward that configuration (OpenGitOps Working Group, n.d.). Continux uses Argo CD in the *App-of-Apps* pattern, in which a root Application points to a directory containing child Applications; this layout lets the entire cluster be bootstrapped and synchronized from a single entry point (Argo Project, n.d.). Scaling Vector during a replay is a deliberate measurement action taken outside the GitOps path, not a replacement for Git as the long-lived configuration source.

Observability is the ability to infer a system's internal condition from external signals such as metrics, logs, and workload state. A *metric* is a numerical *time series* – a sequence of timestamped values representing a quantity over time; metrics are typically *scraped* (pulled on a schedule) by a backend from an HTTP endpoint exposed either by the workload itself or by an *exporter* (a side component that translates internal indicators into a standard metric format). VictoriaMetrics serves as the time-series backend in this artifact, storing metrics over time (VictoriaMetrics, n.d.); Grafana provides dashboard visualization (Grafana Labs, n.d.); and the Continux exporter translates SQL counts and cutover results into standard metrics that VictoriaMetrics scrapes. Dashboards aid interpretation, but reported findings are always cross-checked against actual SQL output and MinIO object listings so that an incomplete or misleading panel is not treated as sole evidence.

2.6 Ursa and related systems

Ursa is a leaderless, Kafka-compatible, lakehouse-native streaming engine that targets cross-AZ replication, disk-storage, and connector cost drivers, while reporting throughput and semantics of its own design (Merli et al., 2025). Continux neither implements Ursa nor compares cost against it; it adds an operational study of GitOps, dashboards, and public analytical-logic replacement in a small deployed stack.

Flink unifies stream and batch processing in one engine (Carbone et al., 2015); Kafka Streams and ksqlDB represent approaches aligned with Kafka ecosystems. RisingWave was chosen here for SQL/MV processing and the MV swap command. This positioning is conceptual and is not a comparative performance experiment.

Table 2.1: Technology layers and evaluated roles

Layer	Technology	Evaluated role
Orchestration	K3s, Argo CD	Run workloads and reconcile manifests
Ingestion	Vector	Controlled JSONL replay
Event broker	Redpanda	Kafka-compatible event topic
Stream compute	RisingWave	Join, aggregate, public/blue/green MVs
Lakehouse storage	MinIO, Iceberg	Checkpoint and output table objects
Observation	VictoriaMetrics, Grafana, exporter	Counts, swap, health, and resources

3. Method and System Architecture

3.1 Research method

This study applies the Design Science Research (DSR) method (Hevner et al., 2004), a methodological framework for information-systems research in which knowledge is produced through the deliberate construction and evaluation of technical artifacts. In DSR, an *artifact* is a designed technical object that addresses a specific problem; here it is the entire Continux architecture comprising the K3s infrastructure, the streaming pipeline from Vector to Iceberg, the GitOps layer using Argo CD, and the observability layer based on VictoriaMetrics and Grafana. *Evaluation* is the phase that examines the artifact through controlled experimental runs rather than merely describing intended configuration on paper.

Evaluation evidence consists of command output as text files, metric responses returned by VictoriaMetrics in JSON form, and Grafana dashboard captures. Output produced on the deployment host (node `imac`) was copied to a Windows analysis workspace at `evidence/finalize/20260522-151720/` so that figures and numbers could be cross-checked while writing the report. Only state, counts, and timings present in evidence are reported as outcomes; properties for which no corresponding measurement exists – including end-to-end exactly-once reconciliation and multi-load benchmarks defined in the repository – are listed explicitly as limitations rather than inferred.

3.2 Dataset and replay preparation

The New York City Taxi and Limousine Commission (NYC TLC) publishes public taxi trip records, one of the most commonly used datasets in urban-mobility analytics research (New York City Taxi and Limousine Commission, n.d.). Continux uses Yellow Taxi data for `2026-03`. A converter script transforms the source Parquet file – a compressed columnar format optimized for batch analytics – into JSONL to produce an ordered event stream for replay. The TLC Taxi Zone lookup table contains `265` rows and is kept as a CSV on MinIO; each row maps a `location_id` to a `borough` and a `zone`.

Table 3.1: Experimental data source

Property	Value
Trip type and month	Yellow Taxi, `2026-03`
Source/output formats	Parquet / JSONL
Confirmed JSONL rows	`3,952,451`
Lookup data	TLC Taxi Zone, `265` rows
Reported run	Controlled replay, final public MV at `986` trips

Each JSONL row is a serialized taxi trip; when Vector emits the row into the broker, it becomes an event in the replay. Vector adds two meta fields on emission: `event_id` (a unique identifier for each emitted event) and `event_time` (the wall-clock moment of emission). The deployed manifest rate-limits replay to `2` events per second so that the resource-constrained lab cluster can process the stream stably; Vector is deliberately stopped after the observation target is reached. Therefore the reported MV is not a full-file processing result; a deeper analysis of raw-data quality is provided in Section 4.3 of Chapter 4.

3.3 Overall architecture

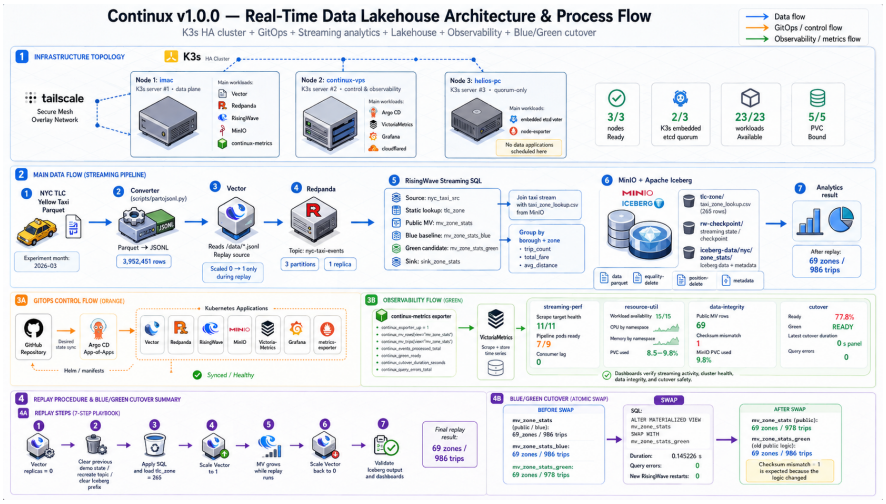

Figure 3.1: Continux architecture and process flow.

Figure 3.1 presents three concurrent paths. In the data path, Parquet is converted to JSONL for event replay, Redpanda buffers the stream, RisingWave maintains SQL results, and the sink writes Iceberg objects on MinIO. In the control path, GitHub holds manifests while Argo CD deploys and reconciles Kubernetes resources. In the observation path, workloads and the exporter expose metrics for VictoriaMetrics storage and Grafana visualization.

Keeping these paths distinct clarifies the measurement boundary: Iceberg objects establish that the data output path ran; Argo CD status establishes the state of core deployed configuration; query-loop and cutover metrics establish whether the public query name remained usable during the logic switch.

3.4 Deployment topology

Table 3.2: Experimental K3s cluster topology

Node	Recorded resources	Role
`imac`	Intel i5-8500, 6 cores, 8 GB RAM, 200 GB SSD, Ubuntu 26.04	Server #1; Vector, Redpanda, RisingWave, MinIO, exporter
`continux-vps`	2 vCPU, 4 GB RAM, 80 GB SSD, Ubuntu 24.04	Server #2; Argo CD, VictoriaMetrics, Grafana
`helios-pc`	WSL Ubuntu 26.04, Intel i5-12500H host, 16 GB RAM	Server #3; quorum-only, node-exporter

Nodes communicate through *Tailscale* – a mesh VPN built on the WireGuard protocol that creates an overlay network among hosts that may be in different physical networks, as if they shared a LAN. K3s in HA mode uses *embedded etcd*: *etcd* is a distributed key-value store that holds the Kubernetes cluster state, and "embedded" means it runs inside the K3s process rather than separately. The cluster requires a `2/3` *quorum*: at least two of three etcd members must remain online for write operations to reach consensus, which is why `helios-pc` is reserved as a quorum-only voter and does not run data workloads. Compute and storage workloads are placed on `imac` to exploit its local SSD and RAM; `continux-vps` hosts the control plane and observability stack.

3.5 SQL objects and Blue/Green design

Table 3.3: Core data objects

Object	Input	Function
`tlc_zone`	CSV on MinIO	Location-to-zone lookup
`nyc_taxi_src`	Redpanda topic	JSON event source
`mv_zone_stats_blue`	Source + lookup	Baseline aggregate
`mv_zone_stats`	Initial blue MV	Public queried name
`mv_zone_stats_green`	Source + lookup + filter	Candidate logic
`sink_zone_stats`	Public MV at sink creation	Iceberg upsert output

The public name `mv_zone_stats` is the client query contract: clients need not modify SQL when serving logic changes from blue to green. Conversely, `sink_zone_stats` was created from a dependency that existed before the swap, so green results at the public MV do not by themselves establish that the Iceberg sink automatically changed logic.

Listing 3.1: Additional filtering in the green logic

```
WHERE t.fare_amount >= 0
  AND t.trip_distance >= 0
```

Once the green MV was populated, a loop queried the public MV at a short interval while the system performed:

Listing 3.2: Blue/Green MV cutover command

```
ALTER MATERIALIZED VIEW mv_zone_stats
SWAP WITH mv_zone_stats_green;
```

3.6 Evaluation measures

Table 3.4: Evaluation measure groups and interpretation

Group	Evidence	Interpretation
Cutover	Duration, query errors, restarts, counts	Public MV switched without observed query error
Observed integrity	MV counts, filter difference, objects	Output exists; difference is explained by logic
Streaming activity	Replay progress, dashboard, displayed lag	Demonstrates activity, not general benchmark
Resource stability	Nodes, workloads, PVC, CPU/RAM	Cluster sustains measured scenario

3.7 Evidence provenance and traceability

The local evidence bundle for the measured run is located at `evidence/finalize/20260522-151720/` and contains `48` textual or JSON output files. Baseline files record node, PVC, Argo CD, and RisingWave readiness; the `05-*` group records replay and Iceberg listing results; and the `06-*` group records green catch-up, the query loop, swap, duration, and metrics stored by VictoriaMetrics.

Evidence interpretation separates event time from later metric retrieval time. The swap timestamp recorded in `06-cutover-duration.txt` is epoch `1779466691`, corresponding to `2026-05-22 16:18:11 UTC`, within the query-loop observation window. VictoriaMetrics files confirm duration and query-error metrics at scrape timestamp `1779467656`, later than the swap; that timestamp establishes persisted observation rather than replacing the cutover time. Metadata in the source bundle shows that the swap/duration files were updated after the query-loop log and post-swap result; the duration is therefore reported as a runbook-recorded measure, not as latency independently reconstructed from an immutable trace.

4. Experimental Results

4.1 Environment and versions

Table 4.1: Software versions recorded in the repository

Component	Version
K3s	`v1.35.5+k3s1`
Helm	`v4.1.4`
Argo CD	`v3.4.2`
Tailscale	`v1.98.2`
Redpanda	`v26.1.8`
RisingWave	`v2.8.3`
Vector	`0.55.0-alpine`
VictoriaMetrics	`v1.143.0`
Grafana	`v13.0.1+security-01`
cloudflared	`2026.5.0`

4.2 Cluster, GitOps, and pipeline readiness

Table 4.2: Infrastructure and pipeline results

Measure	Recorded result
K3s nodes Ready	3/3
PVCs Bound	5/5
Workloads Available after replay	23/23
Main Argo CD applications	Synced/Healthy
RisingWave roles	meta, compute, compactor, frontend RUNNING
TLC lookup table	265 rows
Iceberg objects	data Parquet, equality-delete, position-delete, metadata

4.3 Source data-quality analysis

Before interpreting the replay outcomes, the study examines the raw dataset directly to explain why the green logic was designed as it is. The complete file `data/raw/yellow_tripdata_2026-03.jsonl` is scanned sequentially once on host `imac` using a Python script – without sampling – to count quality violations and compute basic distributions for `fare_amount`, `trip_distance`, `pu_location_id`, and `pickup_time`. Table 4.3 summarizes the result.

Table 4.3: Quality analysis of the NYC TLC Yellow Taxi 2026-03 dataset after JSONL conversion

Metric observed in raw data	Value
Total JSONL records	3,952,451
Records with `fare_amount < 0`	20,724 (0.5243%)
Records with `trip_distance < 0`	0
Records with `fare_amount = 0`	2,995
`fare_amount` min / max	-990.00 / 1,843.30 USD
`fare_amount` mean / median	21.33 / 15.60 USD
`trip_distance` min / max	0 / 288,381.68 miles
`trip_distance` mean / median	5.98 / 1.82 miles
`pu_location_id` values present	261/265 codes
`pickup_time` range	2008-12-31 to 2026-04-01 (35 distinct days)
Peak hour by `pickup_time`	18:00 with 279,487 trips
Lowest hour by `pickup_time`	04:00 with 35,646 trips

The analysis yields three implications for design and evaluation. First, the green logic's `fare_amount >= 0` predicate is grounded in a real phenomenon: 20,724 records carry negative fares, representing 0.5243% of the raw dataset. This small rate is consistent with the observed 8-trip difference between blue (986) and green (978) in the replay sample: at 986 trips, the expected count at that raw-data rate is approximately $986 \times 0.5243\% \approx 5$, near the observed difference given emission order and uneven zone distribution. Second, no record has `trip_distance < 0`; that predicate is preventive for other slices rather than a filter contributing exclusions in this run. Third, the maximum `trip_distance` is 288,381.68 miles and `pickup_time` includes dates outside 2026-03 (from 2008-12-31 to 2026-04-01); the current green logic does not handle these raw-schema anomalies.

The analysis also explains why the MV contains only 69 (`borough`, `zone`) pairs although the dataset touches 261 `pu_location_id` codes: within the measured replay window, Vector delivered a 986-trip sample before it was deliberately stopped, and that sample did not cover every zone. The peak hour at 18:00 and the lowest hour at 04:00 illustrate time-varying ITS load; these dataset-level figures do not represent measured artifact throughput.

4.4 Replay ingestion and lakehouse output

Vector is enabled for a controlled interval and then returned to `replicas=0`. The file `05-mv-before-stop.txt` records the public MV at `66 zones / 912 trips`; after already received events are processed, `05-mv-final-count.txt` records the stable end state of `69 zones / 986 trips`. The file `05-minio-iceberg-after-replay.txt` lists newly produced Iceberg output, showing that the lakehouse output path operated during replay.

This result does not represent peak throughput or full processing of all 3,952,451 JSONL records. The load-profile files in the repository have no accompanying run results and are not reported as a benchmark.

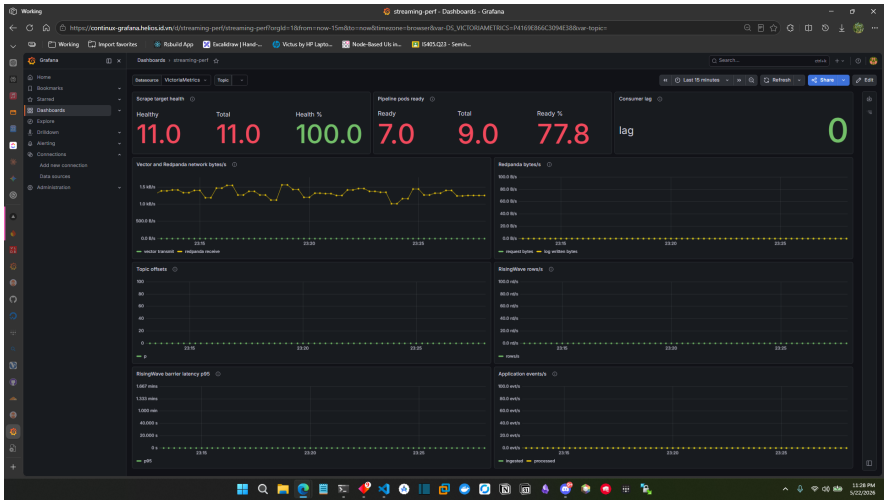

Figure 4.1: Streaming-performance dashboard after the replay/cutover period. Consumer lag displays `0` in the captured window.

4.5 Blue/Green MV cutover

Table 4.4: Blue/Green MV swap results

Measure	Value
Public MV before swap	`69 zones / 986 trips`
Blue MV before swap	`69 zones / 986 trips`
Green MV before swap	`69 zones / 978 trips`
Cutover command	`ALTER MATERIALIZED VIEW ... SWAP WITH ...`
Runbook-recorded duration	`0.145226 s`
Query errors in loop	`0`
RisingWave restarts during swap	`0`
Public MV after swap	`69 zones / 978 trips`
View retaining the green name after swap	`69 zones / 986 trips`

The 8-trip difference between blue and green matches green's exclusion of records with negative `fare_amount` or `trip_distance`. Available evidence does not identify this difference as a transition failure.

The query loop in `06-query-loop-during-cutover.txt` records successful reads transitioning from `69|986` to `69|978` between `16:17:36-16:18:16 UTC` on `2026-05-22`, with no `ERROR` line. The duration file records the swap timestamp at `16:18:11 UTC` and duration `0.145226 s`; VictoriaMetrics files collected afterward confirm that the duration metric and `continux_query_errors_total = 0` were stored. Because the source bundle's swap/duration files have update times after the transition log, the duration is treated as a runbook-recorded measure associated with the observed transition, not independently reconstructed latency. This supports no observed interruption in the measured query loop, not absence of every client-side or network failure mode.

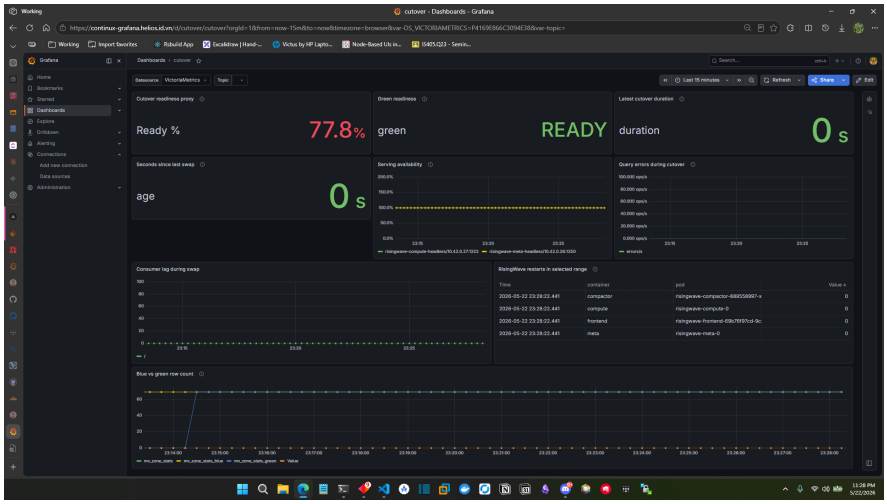

Figure 4.2: Cutover dashboard: green readiness, zero query errors, and zero RisingWave restarts. The duration panel visually rounds a sub-second measurement.

4.6 Observed integrity and resources

The integrity dashboard displays `69` public MV rows and a checksum mismatch of `1` after the swap. The exporter compares public output with baseline logic; once public represents green logic, this mismatch is expected and is not evidence of duplicate or lost records. The "Iceberg freshness" panel displays "56 years"; because timestamp mapping is invalid in the current observation setup, this value is excluded from the findings.

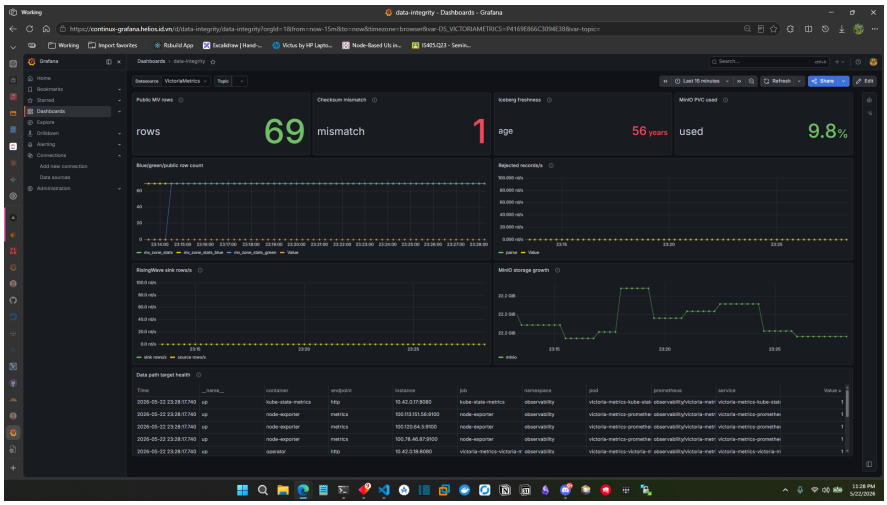

Figure 4.3: Data-integrity dashboard after swap; mismatch is interpreted as a deliberate logic difference.

The resource dashboard displays workload availability of `15/15` at capture time and MinIO/Redpanda PVC usage at approximately `9.8%`. Local output `05-k3s-overview-after-replay.txt` records `23/23` workloads available after replay, approximately `49%` RAM usage on `imac`, and approximately `11%` usage of its root disk.

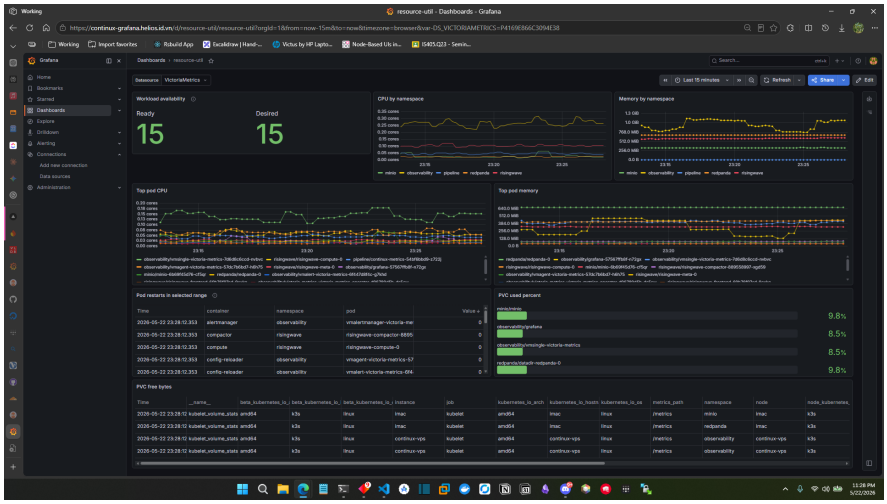

Figure 4.4: Resource-utilization dashboard during the experimental observation window.

4.7 Evaluation summary

Table 4.5: Evidence-bounded findings

Group	Recorded evidence	Conclusion
Query-layer cutover	Runbook duration `0.145226 s`, query errors `0`, RisingWave restarts `0`	Observed zero-downtime for public MV
Lakehouse output	Iceberg objects after replay	Sink wrote output during replay
Logic difference	Green has `8` fewer trips than blue by the filter	Filter change is observable
Cluster stability	`3/3`, `5/5`, `23/23`	Cluster runs measured scenario
End-to-end exactly-once	No reconciliation evidence	Not concluded
Multi-load performance	No completed run results	Not concluded

5. Discussion and Conclusion

5.1 Discussion

RQ1 is answered positively within the observed boundary: the public MV name moved from baseline to candidate logic without a query-loop error. The runbook-recorded duration of `0.145226 s` is an operational indicator for a small SQL-logic change on this laboratory cluster; since file provenance does not enable independent latency reconstruction, it is neither an SLA nor proof for all query patterns or failure modes.

RQ2 is supported by Parquet and metadata objects in MinIO after replay. This demonstrates an operational sink in the exercised pipeline; because RisingWave documents the swap as an MV name operation and treats dependent sinks separately, the study does not claim that the sink moved to green logic after the swap.

RQ3 is supported by cluster health and resource dashboards. The low rate limit and the deliberate shutdown of Vector protect the resource-constrained data-plane node; the result establishes functional feasibility, not maximum capacity.

Relative to Ursa, Continux does not propose a streaming engine. It contributes an operational perspective that combines lakehouse-streaming motivation with GitOps, observability, and public query-logic replacement on a modest deployed environment.

5.2 Limitations

Table 5.1: Evidence limitations and consequences

Limitation	Consequence
No offset, stable-event-key, duplicate, or loss reconciliation	End-to-end exactly-once is not verified
Replay stopped at `986` trips	No full-dataset or peak-throughput claim
No measured runs for three load levels	No load/latency benchmark is reported
Sink dependency predates MV swap	No Iceberg green-cutover claim
Incomplete Kafka catalog/Iceberg freshness metrics	Findings rely on SQL counts and MinIO listings
Duration/swap files updated after the transition log	Duration is reported as a runbook measure, not independently reconstructed latency
Topic replication factor is `1`	Broker node-failure tolerance is not evaluated
Raw dataset can contain `pickup_time` outside the configured month and extreme `trip_distance` outliers	Current green logic filters only negative values; time-range and distance-range filters are not applied

5.3 Future work

Future research can pursue four complementary directions. First, attach a stable *event identity* – an identifier preserved from the source JSONL through to the Iceberg sink – so that duplicates and losses can be reconciled across the entire path, allowing end-to-end exactly-once to be verified by measurement rather than inferred from connector configuration. Second, run measured multi-load benchmarks on a larger data plane, using the load profiles defined in the repository (`low`, `medium`, `high`) to produce latency/throughput curves. Third, design a cutover that also covers the downstream sink – for instance creating a new sink bound to the green MV before cutover and renaming the sink, or compacting and rewriting Iceberg snapshots under the new logic – so that conclusions can be extended to the full lakehouse path. Fourth, extend the source from a recorded dataset to live mobility events (for example traffic sensors or bus GPS telemetry). These are directions for subsequent releases, not part of the current results.

5.4 Conclusion

Continux successfully implements a real-time data lakehouse artifact on K3s comprising ingestion, event brokering, streaming SQL, Iceberg object storage, GitOps, and observability. From available evidence, one controlled replay produced a public MV and Iceberg output; during a Blue/Green MV swap, the query loop recorded `0` errors with no RisingWave restart, and the runbook recorded a measured duration of `0.145226 s`.

Its principal experimental contribution is evidence that public-query logic can be updated without observed interruption during the measured cutover window. The report intentionally refrains from claiming verified end-to-end exactly-once behavior, multi-load benchmarking, or full-path sink cutover without corresponding measurements.

References

Apache Software Foundation. (n.d.). *Apache iceberg documentation*. Retrieved May 24, 2026, from https://iceberg.apache.org/docs/latest/

Argo Project. (n.d.). *Argo cd documentation*. Retrieved May 24, 2026, from https://argo-cd.readthedocs.io/

Armbrust, M., Ghodsi, A., Xin, R., & Zaharia, M. (2021). Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics. *Proceedings of the 11th Conference on Innovative Data Systems Research (CIDR)*. https://www.vldb.org/cidrdb/2021/lakehouse-a-new-generation-of-open-platforms-that-unify-data-warehousing-and-advanced-analytics.html

Carbone, P., Katsifodimos, A., Ewen, S., Markl, V., Haridi, S., & Tzoumas, K. (2015). Apache flink: Stream and batch processing in a single engine. *IEEE Data Engineering Bulletin*, *38*(4), 28–38. https://sites.computer.org/debull/A15dec/p28.pdf

Grafana Labs. (n.d.). *Grafana documentation*. Retrieved May 24, 2026, from https://grafana.com/docs/grafana/latest/

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, *28*(1), 75–105. https://doi.org/10.2307/25148625

Kreps, J., Narkhede, N., & Rao, J. (2011). Kafka: A distributed messaging system for log processing. *Proceedings of the NetDB Workshop*. https://cwiki.apache.org/confluence/download/attachments/27822226/Kafka-netdb-06-2011.pdf

Merli, M., Guo, S., Li, P., Chen, H., & Lu, N. (2025). Ursa: A lakehouse-native data streaming engine for kafka. *Proceedings of the VLDB Endowment*, *18*(12), 5184–5196. https://doi.org/10.14778/3750601.3750636

New York City Taxi and Limousine Commission. (n.d.). *Tlc trip record data*. Retrieved May 24, 2026, from https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page

OpenGitOps Working Group. (n.d.). *Opengitops principles*. Retrieved May 24, 2026, from https://opengitops.dev/

Redpanda Data. (n.d.). *Redpanda documentation*. Retrieved May 24, 2026, from https://docs.redpanda.com/

RisingWave Labs. (n.d.-a). *Alter a streaming job*. Retrieved May 24, 2026, from https://docs.risingwave.com/operate/alter-streaming

RisingWave Labs. (n.d.-b). *Alter materialized view: Swap with*. Retrieved May 24, 2026, from https://docs.risingwave.com/sql/commands/sql-alter-materialized-view

RisingWave Labs. (n.d.-c). *Deliver data to apache iceberg tables*. Retrieved May 24, 2026, from https://docs.risingwave.com/iceberg/deliver-to-iceberg

United Nations. (n.d.-a). *Goal 11: Sustainable cities and communities*. Retrieved May 24, 2026, from https://sdgs.un.org/goals/goal11

United Nations. (n.d.-b). *Goal 8: Decent work and economic growth*. Retrieved May 24, 2026, from https://sdgs.un.org/goals/goal8

United Nations. (n.d.-c). *Goal 9: Industry, innovation and infrastructure*. Retrieved May 24, 2026, from https://sdgs.un.org/goals/goal9

VictoriaMetrics. (n.d.). *Victoriametrics documentation*. Retrieved May 24, 2026, from https://docs.victoriametrics.com/

A. Core SQL

A.1 Source and baseline view

```
CREATE SOURCE IF NOT EXISTS nyc_taxi_src (
    event_id VARCHAR,
    event_time TIMESTAMPTZ,
    pickup_time TIMESTAMPTZ,
    pu_location_id INT,
    do_location_id INT,
    fare_amount DOUBLE PRECISION,
    trip_distance DOUBLE PRECISION
) WITH (
    connector = 'kafka',
    topic = 'nyc-taxi-events',
    properties.bootstrap.server = 'redpanda.redpanda:9093',
    scan.startup.mode = 'earliest'
) FORMAT PLAIN ENCODE JSON;

CREATE MATERIALIZED VIEW IF NOT EXISTS mv_zone_stats_blue AS
SELECT z.borough, z.zone, COUNT(*) AS trip_count,
       SUM(t.fare_amount) AS total_fare,
       AVG(t.trip_distance) AS avg_distance
FROM nyc_taxi_src t
JOIN tlc_zone z ON t.pu_location_id = z.location_id
GROUP BY z.borough, z.zone;

CREATE MATERIALIZED VIEW IF NOT EXISTS mv_zone_stats AS
SELECT * FROM mv_zone_stats_blue;
```

A.2 Green view and swap

```
CREATE MATERIALIZED VIEW IF NOT EXISTS mv_zone_stats_green AS
SELECT z.borough, z.zone, COUNT(*) AS trip_count,
       SUM(t.fare_amount) AS total_fare,
       AVG(t.trip_distance) AS avg_distance
FROM nyc_taxi_src t
JOIN tlc_zone z ON t.pu_location_id = z.location_id
WHERE t.fare_amount >= 0
  AND t.trip_distance >= 0
GROUP BY z.borough, z.zone;

ALTER MATERIALIZED VIEW mv_zone_stats
SWAP WITH mv_zone_stats_green;
```

A.3 Iceberg sink

```
CREATE SINK IF NOT EXISTS sink_zone_stats FROM mv_zone_stats
WITH (
    connector = 'iceberg',
    type = 'upsert',
    primary_key = 'borough,zone',
    catalog.type = 'storage',
    warehouse.path = 's3://iceberg-data/',
    database.name = 'nyc',
    create_table_if_not_exists = 'true',
    table.name = 'zone_stats'
);
```

B. Operational Configuration Excerpts

```
# Redpanda topic
topics:
  - name: nyc-taxi-events
    partitions: 3
    replicationFactor: 1
    config:
      retention.ms: "86400000"
```

```
# Vector rate in the deployed ConfigMap
rate_limit_num = 2
rate_limit_duration_secs = 1

# RisingWave state store
stateStore:
  s3:
    enabled: true
    endpoint: http://minio.minio.svc.cluster.local:9000
    bucket: rw-checkpoint
```